TP ANDROID

Liste de contacts - Version 2

COMPTE RENDU

Réalisé par : Mjahed Hanin

Classe : L3MDW1

Date : 04/11/2026

Dépôt GitHub :

https://github.com/haninemjahed-lgtm/Liste_de_contactsv2

Kotlin - Android Studio - Interfaces XML

1. Introduction

L'objectif de ce TP est de réaliser une application Android de gestion de contacts en Kotlin. L'application comporte deux activités : **MainActivity**, qui affiche la liste des contacts dans une ListView personnalisée, et **DetailActivity**, qui permet de consulter et modifier un contact, de l'appeler, de lui envoyer un e-mail et de changer sa photo (galerie ou appareil photo).

Outils utilisés : Android Studio, Kotlin, ViewBinding, ListView avec BaseAdapter, Activity Result API, émulateur Pixel 7 (API 36), Git et GitHub.

2. Travail réalisé

Modèle de données

Création de la classe **Contact** (data class Serializable) avec id, nom, prénom, téléphone, e-mail et chemin de la photo (nullable). Le caractère Serializable permet de transmettre l'objet dans un Intent, et la méthode copy() évite de modifier l'original avant l'enregistrement.

Liste personnalisée

Création du layout **item_contact.xml** (photo, nom et prénom, téléphone) et de la classe **ContactAdapter** (BaseAdapter). Cinq contacts fictifs sont affichés dans la ListView de MainActivity. Une icône par défaut est utilisée quand aucune photo n'est définie.

Intent explicite

Au clic sur un contact, MainActivity lance DetailActivity avec un Intent explicite contenant le contact et sa position dans la liste.

Enregistrer et Annuler

DetailActivity renvoie le contact modifié avec setResult(RESULT_OK). MainActivity le reçoit grâce à l'Activity Result API (registerForActivityResult), remplace l'élément et appelle notifyDataSetChanged() : la liste est actualisée immédiatement. Le bouton Annuler renvoie RESULT_CANCELED sans rien modifier. Les champs nom, prénom et téléphone sont obligatoires.

Intents implicites

Le bouton Appeler utilise ACTION_DIAL (ouverture du composeur sans permission) et le bouton e-mail utilise ACTION_SENDTO avec le schéma mailto:. Un bloc try/catch sur ActivityNotFoundException affiche un message si aucune application compatible n'existe.

Galerie

Le contrat GetContent() ouvre le sélecteur d'images. L'image choisie est copiée dans le stockage interne (filesDir) pour que l'accès reste valide, et son chemin est enregistré dans le contact. Une sélection annulée (Uri nul) affiche un message.

Appareil photo

Utilisation de TakePicture() avec un FileProvider, de la permission CAMERA demandée à l'exécution (RequestPermission) et de la vérification FEATURE_CAMERA_ANY. Les trois cas particuliers sont gérés : permission refusée, caméra indisponible et prise de photo annulée.

3. Tests effectués

Tous les tests ont été réalisés sur l'émulateur Pixel 7 (Android 16, API 36). Les captures ci-dessous proviennent de l'application développée.

Test 1 : Affichage de MainActivity

La liste affiche les cinq contacts avec leur photo par défaut, leur nom, leur prénom et leur numéro de téléphone.

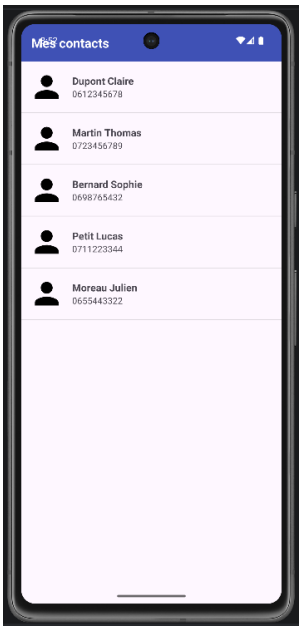


Figure 1 : MainActivity

Test 2 : Ouverture de DetailActivity

Un clic sur le contact Martin Thomas ouvre DetailActivity (Intent explicite) avec ses informations dans les champs modifiables.



Figure 2 : DetailActivity

Test 3 : Modification puis enregistrement

Le nom est changé en « Mjahed » puis le bouton Enregistrer est utilisé. La liste se met à jour immédiatement avec le nouveau nom.

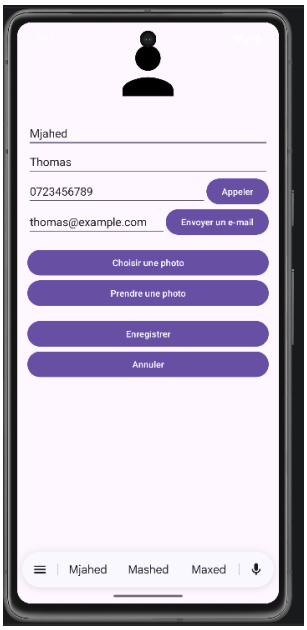


Figure 3 : nom modifié

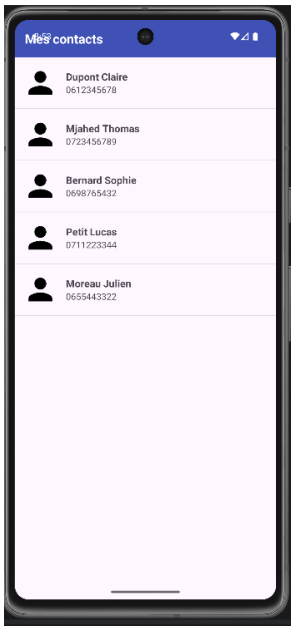


Figure 4 : liste mise à jour

Test 4 : Annulation d'une modification

Le nom est modifié dans DetailActivity (« moujahed ») puis le bouton Annuler est utilisé : on revient à la liste sans enregistrer et les anciennes données sont conservées.



Figure 5 : modification avant Annuler

Test 5 : Champ obligatoire vide

Le champ Nom est vidé puis Enregistrer est pressé. Le message « Champ obligatoire » s'affiche et l'écran reste ouvert.



Figure 6 : champ Nom vidé

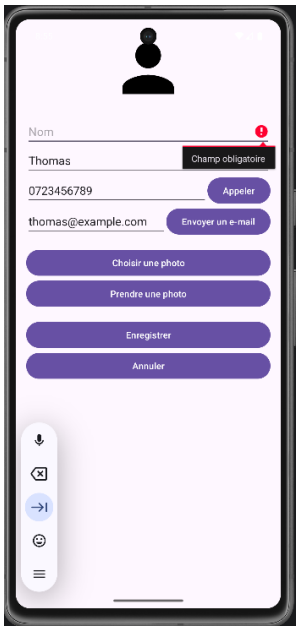


Figure 7 : message d'erreur

Test 6 : Appel et e-mail (Intents implicites)

Le bouton Appeler ouvre le composeur téléphonique avec le numéro du contact. Le bouton d'e-mail ouvre l'application Gmail de l'émulateur (écran de bienvenue au premier lancement).

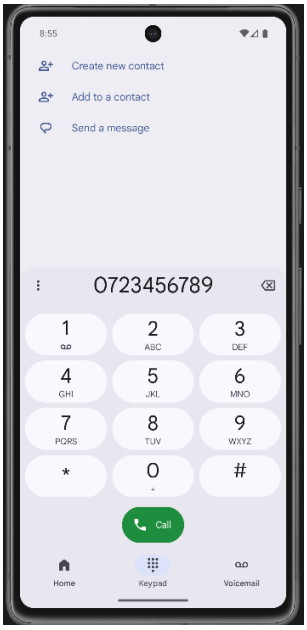


Figure 8 : composeur avec le numéro



Figure 9 : application Gmail ouverte

Test 7 : Choix d'une photo dans la galerie

Le bouton « Choisir une photo » ouvre le sélecteur d'images. Après sélection et enregistrement, la photo apparaît dans la liste.

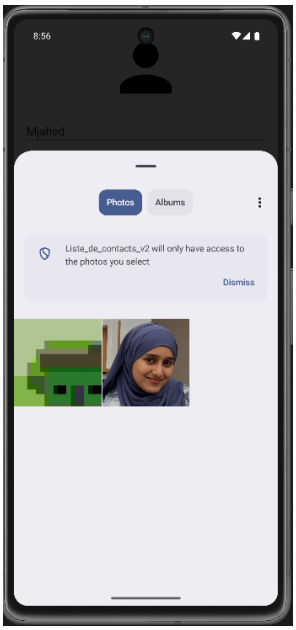


Figure 10 : sélecteur de la galerie

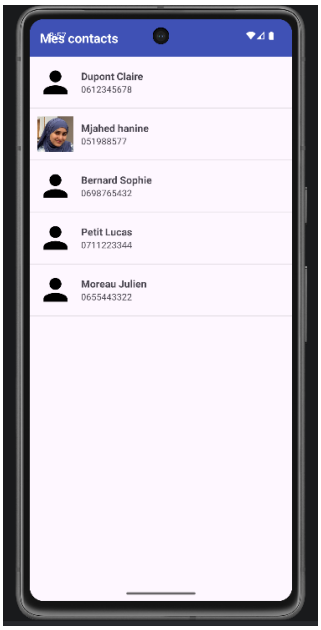


Figure 11 : photo visible dans la liste

Test 8 : Annulation de la galerie

Un retour dans le sélecteur sans choisir d'image affiche le message « Sélection annulée » et la photo actuelle est conservée.

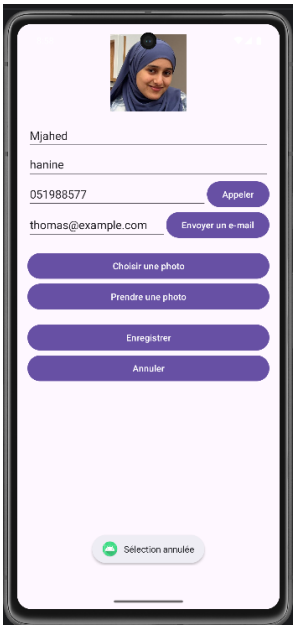


Figure 12 : sélection annulée

Test 9 : Permission de la caméra

Au clic sur « Prendre une photo », Android demande l'autorisation d'utiliser la caméra. En cas de refus, le message « Permission caméra refusée » s'affiche.

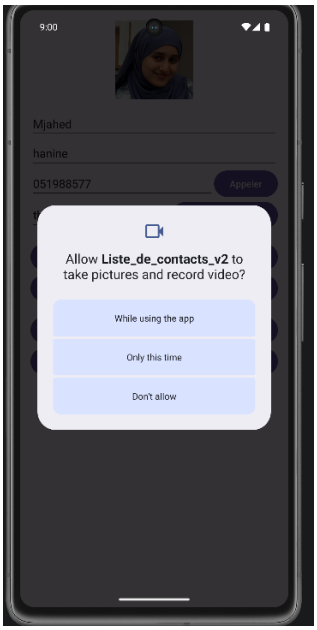


Figure 13 : demande de permission

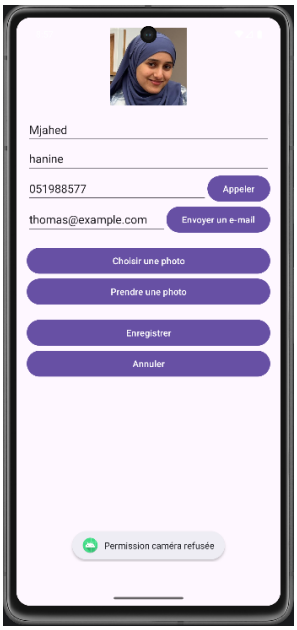


Figure 14 : permission refusée

Test 10 : Prise de photo avec la caméra

Après avoir autorisé la caméra, la photo est prise puis s'affiche dans DetailActivity.



Figure 15 : caméra de l'émulateur

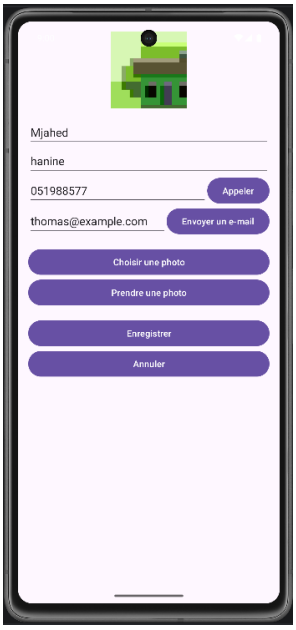


Figure 16 : photo affichée

Test 11 : Annulation de la prise de photo

Un retour depuis la caméra sans valider affiche le message « Prise de photo annulée ». La photo précédente reste affichée.

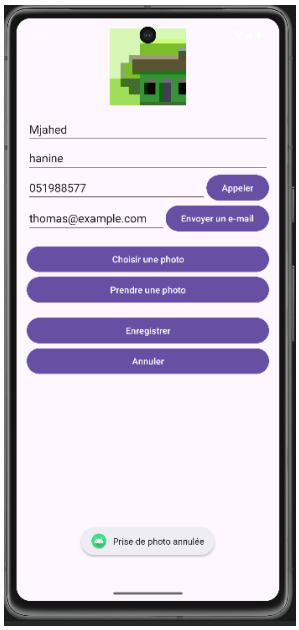


Figure 17 : prise de photo annulée

Récapitulatif

Fonctionnalité testée	Résultat	Figure
Liste personnalisée (photo, nom, prénom, téléphone)	OK	1
Ouverture du détail (Intent explicite)	OK	2
Modification et actualisation immédiate de la liste	OK	3, 4
Annulation sans enregistrement	OK	5
Champs obligatoires vides	OK	6, 7
Appel (Intent implicite)	OK	8
E-mail (Intent implicite)	OK (application ouverte)	9
Choix d'une photo dans la galerie	OK	10, 11
Galerie annulée	OK	12
Permission caméra refusée	OK	13, 14
Photo prise avec la caméra	OK	15, 16
Prise de photo annulée	OK	17

4. Difficultés rencontrées et solutions

Émulateur à l'écran noir

Après le lancement, l'émulateur affichait un écran noir alors que le Logcat ne montrait aucune erreur de l'application (seulement des messages du système et de Google Play Services). Un redémarrage de l'émulateur (Cold Boot) a résolu le problème.

Sélecteur de photos vide

Le sélecteur affichait « No photos yet » car l'émulateur ne contenait aucune image. J'ai ajouté des images dans l'émulateur avant de tester la galerie.

Adresse e-mail incorrecte

Un contact avait une adresse sans point (« gmailcom »), ce qui pouvait fausser le test d'e-mail. L'adresse a été corrigée.

Accès aux photos de la galerie

L'adresse (Uri) renvoyée par la galerie est temporaire. Pour garder la photo, l'image est copiée dans le stockage interne de l'application et c'est ce chemin qui est enregistré.

Caméra : FileProvider et permission

La caméra nécessite la permission CAMERA demandée à l'exécution et un FileProvider déclaré dans le Manifest pour donner à l'application caméra un accès au fichier de destination.

5. Contenu du dépôt GitHub

Le dépôt contient le projet Android Studio complet : **app/** (code Kotlin : MainActivity, DetailActivity, Contact, ContactAdapter ; layouts XML ; AndroidManifest.xml avec la permission CAMERA et le FileProvider), le dossier **captures/** (17 captures d'écran réelles de l'application) et ce compte rendu au format PDF.

6. Conclusion

Ce TP m'a permis de mettre en pratique la communication entre activités avec les Intents explicites et implicites, le retour de résultat avec l'Activity Result API, la création d'une ListView personnalisée avec un adaptateur, ainsi que la gestion des permissions et des fichiers pour la galerie et la caméra. Toutes les fonctionnalités demandées ont été testées sur l'émulateur, y compris les cas particuliers (champs vides, annulation, permission refusée).